MANIPAL
ACADEMY of HIGHER EDUCATION
*(Institution of Eminence Deemed to be University)*

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

# Cryptography-Lab 7

| Reg. Number | Name | Branch |
|---|---|---|
| 261100690016 | Sanjana Rao U S | Cyber Security |



261100690016_Sanjana Rao U S

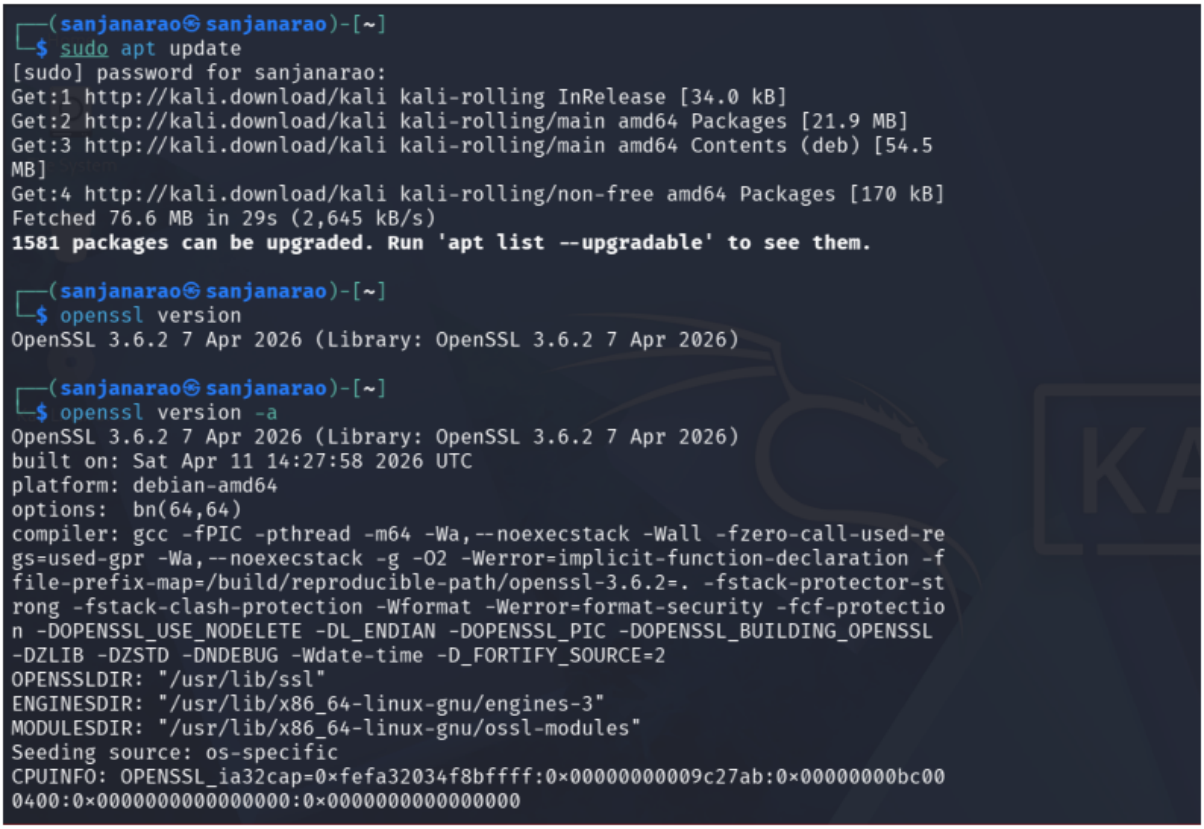
**OpenSSL Tool – Practical Uses in Cryptography**

**1. Introduction**

OpenSSL is an open-source tool used for cryptography and network security. It is available on operating systems such as Kali Linux and Ubuntu. It helps users encrypt and decrypt files, generate public and private keys, calculate hash values, create digital signatures, and manage digital certificates. It is widely used by cybersecurity professionals to protect sensitive information and secure network communication.

**2. Installation and Version Checking**

OpenSSL is generally available in Kali Linux. If it is not installed, it can be installed using the following commands.




The terminal displays the installed OpenSSL version and its configuration details.

**3. Practical Experiments**

Experiment 1: Generate Random Numbers

Aim: To generate random values using OpenSSL.
OpenSSL can generate random bytes that are useful for creating cryptographic keys, security tokens, and other security-related values.



2610069700.16_Sanjana Rao U S

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl rand -hex 16
1d2ccec83c5eee444e833a875a06e604
```

The rand command generates random bytes, while -hex displays them in hexadecimal format. The number 16 specifies that 16 random bytes should be generated.
Result: OpenSSL generates a random hexadecimal value containing 32 characters.

Experiment 2: Perform SHA-256 Hashing

Aim: To calculate the SHA-256 hash of a file.
Hashing converts data into a fixed-length value. It is mainly used to verify file integrity and identify whether information has been modified.

Step 1: Create a text file.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo "Hello Cybersecurity" > message.txt
```

Step 2: Generate the SHA-256 hash.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl dgst -sha256 message.txt
SHA2-256(message.txt)= b0ba7d7284c3373619c3c8a3d98d761eab3f0efc76484ef1f9b9a4045a9e4a8e
```

The dgst command calculates the hash value, and -sha256 selects the SHA-256 algorithm.

Step 3: Modify the file.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo "Hello Network Security" > message.txt
```

Step 4: Calculate the hash again.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl dgst -sha256 message.txt
SHA2-256(message.txt)= ffa46ddddfa9e954e0e0e82a8ef9ee4a8d4d08ef98eaf3444bbcfa9f6a69de7c
```

Result: The hash values before and after modifying the file will be different. This demonstrates how hashing helps identify changes in data.

Experiment 3: Encrypt and Decrypt a File Using AES-256

Aim: To encrypt and decrypt a file using symmetric encryption.
AES (Advanced Encryption Standard) is a symmetric encryption algorithm that uses the same secret password for encryption and decryption. It is commonly used to protect confidential files and sensitive information.
Step 1: Create a message file.



2610069700.16_Sanjana Rao U S

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo "This is a confidential cybersecurity message" > secret.txt
```

Step 2: Encrypt the file.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl enc -aes-256-cbc -salt -pbkdf2 -in secret.txt -out encrypted.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
```

OpenSSL asks for a password. Enter a password and confirm it. The command uses AES-256-CBC encryption and PBKDF2 to derive an encryption key from the password.

Step 3: View the encrypted file.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ cat encrypted.enc
Salted__••Vt•
          •d••y•• •e@•┘"+z•;•vc_••• 1•••.Z•D•ɲ•$X••
```

The contents will appear unreadable because the original message has been encrypted.

Step 4: Decrypt the file.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl enc -d -aes-256-cbc -pbkdf2 -in encrypted.enc -out decrypted.txt
enter AES-256-CBC decryption password:
```

Enter the same password used during encryption.

Step 5: Display the decrypted message.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ cat decrypted.txt
This is a confidential cybersecurity message
```

Result: The original message is recovered successfully after decryption.

Experiment 4: Generate RSA Public and Private Keys

Aim: To generate RSA public and private keys using OpenSSL.
RSA is an asymmetric encryption algorithm that uses two related keys. The public key can be shared with others, whereas the private key must be kept secret. These keys are used in encryption, decryption, and digital signatures.



2610069700.16_Sanjana Rao U S

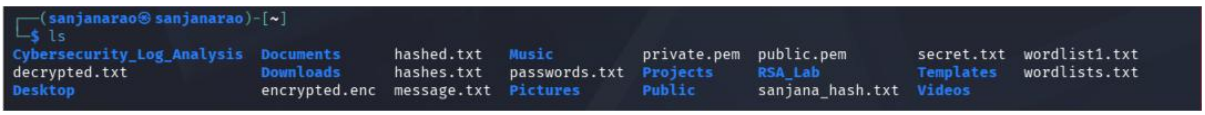
Step 1: Generate a 2048-bit RSA private key.




Step 2: Generate the public key.




Step 3: Display the generated files.




Step 4: Examine the public key.




The genpkey command creates the private key, while the pkey command extracts the corresponding public key. The generated keys are saved in PEM format.
Result: Two files named private.pem and public.pem are created successfully.

Experiment 5: Encrypt and Decrypt Using RSA

Aim: To perform asymmetric encryption and decryption using RSA keys.
RSA encryption allows a sender to encrypt a message using the receiver's public key. The receiver can decrypt the message using the corresponding private key.

Step 1: Create a message file.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo "This message is protected using RSA encryption" > message.txt
```

Step 2: Encrypt the message using the public key.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out encrypted.bin -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

Step 3: Decrypt the message using the private key.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl pkeyutl -decrypt -inkey private.pem -in encrypted.bin -out decrypted.txt -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

Step 4: Display the decrypted message.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ cat decrypted.txt
This message is protected using RSA encryption
```

The pkeyutl command performs public-key operations. The -encrypt option encrypts the message, while -decrypt recovers the original information. OAEP with SHA-256 provides secure padding for RSA encryption.
Result: The original message is successfully recovered using the private key.

Experiment 6: Create and Verify a Digital Signature

Aim: To create and verify a digital signature using OpenSSL.
A digital signature is used to verify the authenticity and integrity of information. The sender signs a document using a private key, and the receiver verifies the signature using the corresponding public key.

Step 1: Create a document.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo "This document is approved by the administrator" > document.txt
```

Step 2: Create a digital signature.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl dgst -sha256 -sign private.pem -out signature.bin document.txt
```

Step 3: Verify the digital signature.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl dgst -sha256 -verify public.pem -signature signature.bin document.txt
Verified OK
```



2610069010.16_Sanjana Rao U S

Step 4: Modify the document.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo "The document has been modified" > document.txt
```

Step 5: Verify the signature again.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl dgst -sha256 -verify public.pem -signature signature.bin document.txt
Verification failure
40374090CB7F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
40374090CB7F0000:error:1C880004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/signature/rsa_sig.c:1038:
```

Result: The signature verification succeeds for the original document and fails after modification. This demonstrates how digital signatures detect unauthorized changes.

Experiment 7: Generate a Self-Signed Digital Certificate
Aim: To create and examine a digital certificate using OpenSSL.
Digital certificates are used to establish trust between websites, servers, and users. OpenSSL can generate self-signed certificates for testing and learning purposes.

Step 1: Generate a private key.

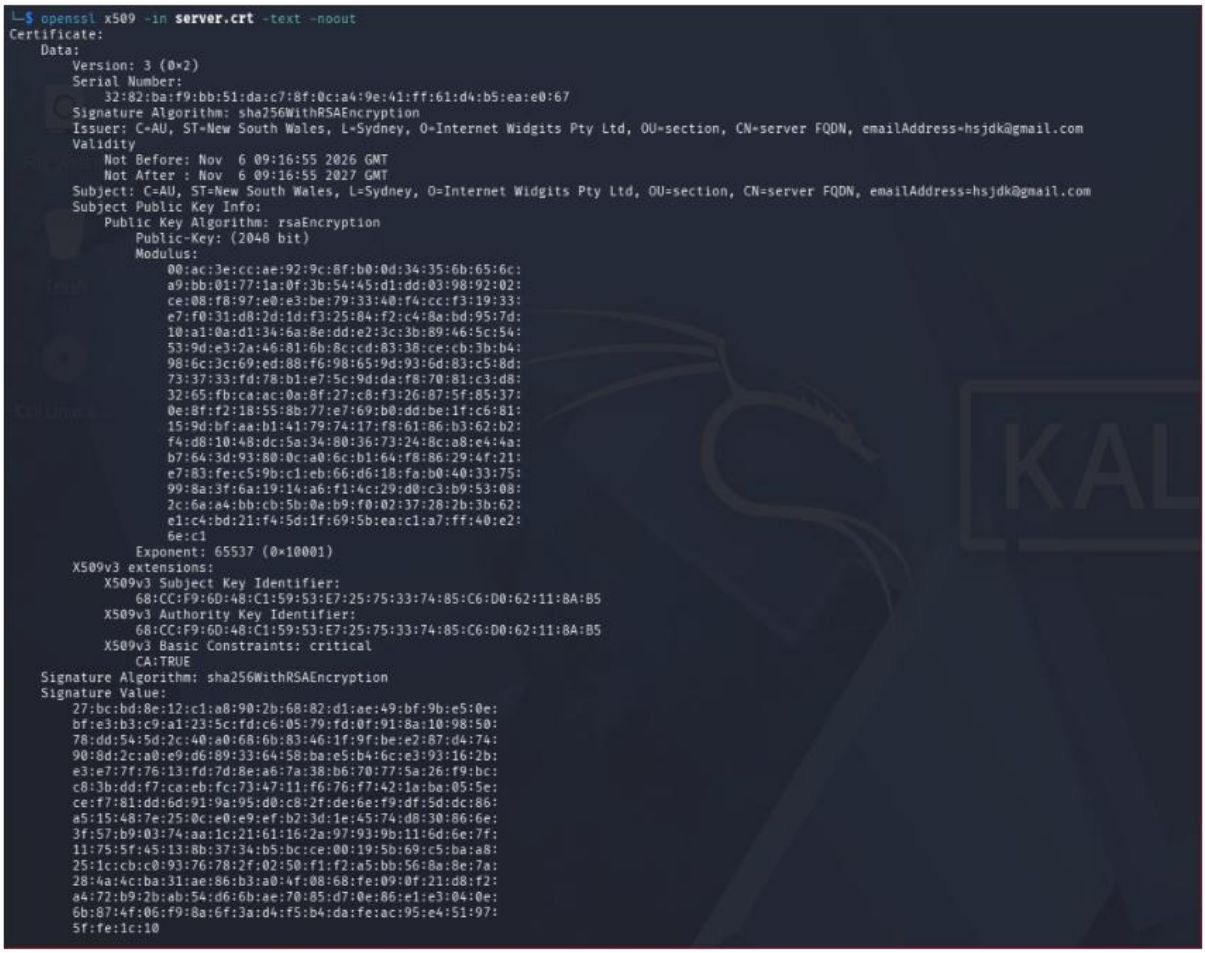



Step 2: Generate a self-signed certificate.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ openssl req -new -x509 -key server.key -out server.crt -days 365
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:AU
State or Province Name (full name) [Some-State]:New South Wales
Locality Name (eg, city) []:Sydney
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Internet Widgits Pty Ltd
Organizational Unit Name (eg, section) []:section
Common Name (e.g. server FQDN or YOUR name) []:server FQDN
Email Address []:hsjdk@gmail.com
```

Enter the requested certificate information. For a local testing example, enter localhost as the Common Name.

Step 3: Display the certificate details.



```
└─$ openssl x509 -in server.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            32:82:ba:f9:bb:51:da:c7:8f:0c:a4:9e:41:ff:61:d4:b5:ea:e0:67
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=AU, ST=New South Wales, L=Sydney, O=Internet Widgits Pty Ltd, OU=section, CN=server FQDN, emailAddress=hsjdk@gmail.com
        Validity
            Not Before: Nov  6 09:16:55 2026 GMT
            Not After : Nov  6 09:16:55 2027 GMT
        Subject: C=AU, ST=New South Wales, L=Sydney, O=Internet Widgits Pty Ltd, OU=section, CN=server FQDN, emailAddress=hsjdk@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:ac:3e:cc:ae:92:9c:8f:b0:0d:34:35:6b:65:6c:
                    a9:bb:01:77:1a:0f:3b:54:45:d1:dd:03:98:92:02:
                    ce:08:f8:97:e0:e3:be:79:33:40:f4:cc:f3:19:33:
                    e7:f0:31:d8:2d:1d:f3:25:84:f2:c4:8a:bd:95:7d:
                    10:a1:0a:d1:34:6a:8e:dd:e2:3c:3b:89:46:5c:54:
                    53:9d:e3:2a:46:81:6b:8c:cd:83:38:ce:cb:3b:b4:
                    98:6c:3c:69:ed:88:f6:98:65:9d:93:6d:83:c5:8d:
                    73:37:33:fd:78:b1:e7:5c:9d:da:f8:70:81:c3:d8:
                    32:65:fb:ca:ac:0a:8f:27:c8:f3:26:87:5f:85:37:
                    0e:8f:f2:18:55:8b:77:e7:69:b0:dd:be:1f:c6:81:
                    15:9d:bf:aa:b1:41:79:74:17:f8:61:86:b3:62:b2:
                    f4:d8:10:4b:dc:5a:34:80:36:73:24:8c:a8:e4:4a:
                    b7:64:3d:93:80:0c:a0:6c:b1:64:f8:86:29:4f:21:
                    e7:83:fe:c5:9b:c1:eb:66:d6:18:fa:b0:40:33:75:
                    99:8a:3f:6a:19:14:a6:f1:4c:29:d0:c3:b9:53:08:
                    2c:6a:a4:bb:cb:5b:0a:b9:f0:02:37:28:2b:3b:62:
                    e1:c4:bd:21:f4:5d:1f:69:5b:ea:c1:a7:ff:40:e2:
                    6e:c1
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                68:CC:F9:6D:48:C1:59:53:E7:25:75:33:74:85:C6:D0:62:11:8A:B5
            X509v3 Authority Key Identifier:
                68:CC:F9:6D:48:C1:59:53:E7:25:75:33:74:85:C6:D0:62:11:8A:B5
            X509v3 Basic Constraints: critical
                CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        27:bc:bd:8e:12:c1:a8:90:2b:68:82:d1:ae:49:bf:9b:e5:0e:
        bf:e3:b3:c9:a1:23:5c:fd:c6:05:79:fd:0f:91:8a:10:98:50:
        78:dd:54:5d:2c:40:a0:68:6b:83:46:1f:9f:be:e2:87:d4:74:
        90:8d:2c:a0:e9:d6:89:33:64:58:ba:e5:b4:6c:e3:93:16:2b:
        e3:e7:7f:76:13:fd:7d:8e:a6:7a:38:b6:70:77:5a:26:f9:bc:
        c8:3b:dd:f7:ca:eb:fc:73:47:11:f6:76:f7:42:1a:ba:05:5e:
        ce:f7:81:dd:6d:91:9a:95:d0:c8:2f:de:6e:f9:df:5d:dc:86:
        a5:15:48:7e:25:0c:e0:e9:ef:b2:3d:1e:45:74:d8:30:86:6e:
        3f:57:b9:03:74:aa:1c:21:61:16:2a:97:93:9b:11:6d:6e:7f:
        11:75:5f:45:13:8b:37:34:b5:bc:ce:00:19:5b:69:c5:ba:a8:
        25:1c:cb:c0:93:76:78:2f:02:50:f1:f2:a5:bb:56:8a:8e:7a:
        28:4a:4c:ba:31:ae:86:b3:a0:4f:08:68:fe:09:0f:21:d8:f2:
        a4:72:b9:2b:ab:54:d6:6b:ae:70:85:d7:0e:86:e1:e3:04:0e:
        6b:87:4f:06:f9:8a:6f:3a:d4:f5:b4:da:fe:ac:95:e4:51:97:
        5f:fe:1c:10
```

The req command creates certificate requests, while -x509 generates a self-signed certificate. The -days 365 option sets the certificate validity period to 365 days.
Result: A self-signed certificate named server.crt is generated and its information is displayed.

Experiment 8: Examine an HTTPS Website Certificate

Aim: To examine the TLS certificate of an HTTPS website.
OpenSSL can connect to an HTTPS server and display information about its TLS connection and digital certificate. This is useful for understanding how secure communication works between websites and users.

Step 1: Connect to an HTTPS website.



2610069010_Sanjana Rai CS




Step 2: Display important certificate information.

```
┌──(sanjanarao㉿sanjanarao)-[~]
└─$ echo | openssl s_client -connect google.com:443 -servername google.com 2>/dev/null | openssl x509 -noout -subject -issuer -dates
subject=CN=*.google.com
issuer=C=US, O=Google Trust Services, CN=WE2
notBefore=Oct  7 19:07:42 2026 GMT
notAfter=Dec 30 19:07:41 2026 GMT
```

The s_client command establishes a TLS connection with the server. The -connect option specifies the website and port, while -servername sends the hostname for Server Name Indication. The x509 command displays the certificate owner, issuing authority, and validity dates.

Result: OpenSSL displays certificate information obtained from the HTTPS website.

**4. Advantages of OpenSSL**

OpenSSL is freely available and supports several cryptographic algorithms. It provides encryption, hashing, key generation, digital signatures, and certificate management. It works on different operating systems and allows users to perform security operations directly through the command line. It is also useful for cybersecurity education, testing, and network security analysis.

## 5. Limitations of OpenSSL

OpenSSL commands can be difficult for beginners because they require an understanding of cryptographic concepts and command options. Losing an encryption password or private key may make protected information inaccessible. Incorrect configurations and poor key management can also reduce security. Self-signed certificates are useful for testing but are not automatically trusted by browsers.

## 6. Conclusion

OpenSSL is an important tool in cryptography and cybersecurity. Through these practical experiments, different operations such as hashing, AES encryption and decryption, RSA key generation, digital signatures, certificate creation, and TLS certificate examination were performed. These experiments help us understand how cryptographic techniques are used in real-world applications to protect sensitive information, maintain data integrity, authenticate messages, and secure network communication.